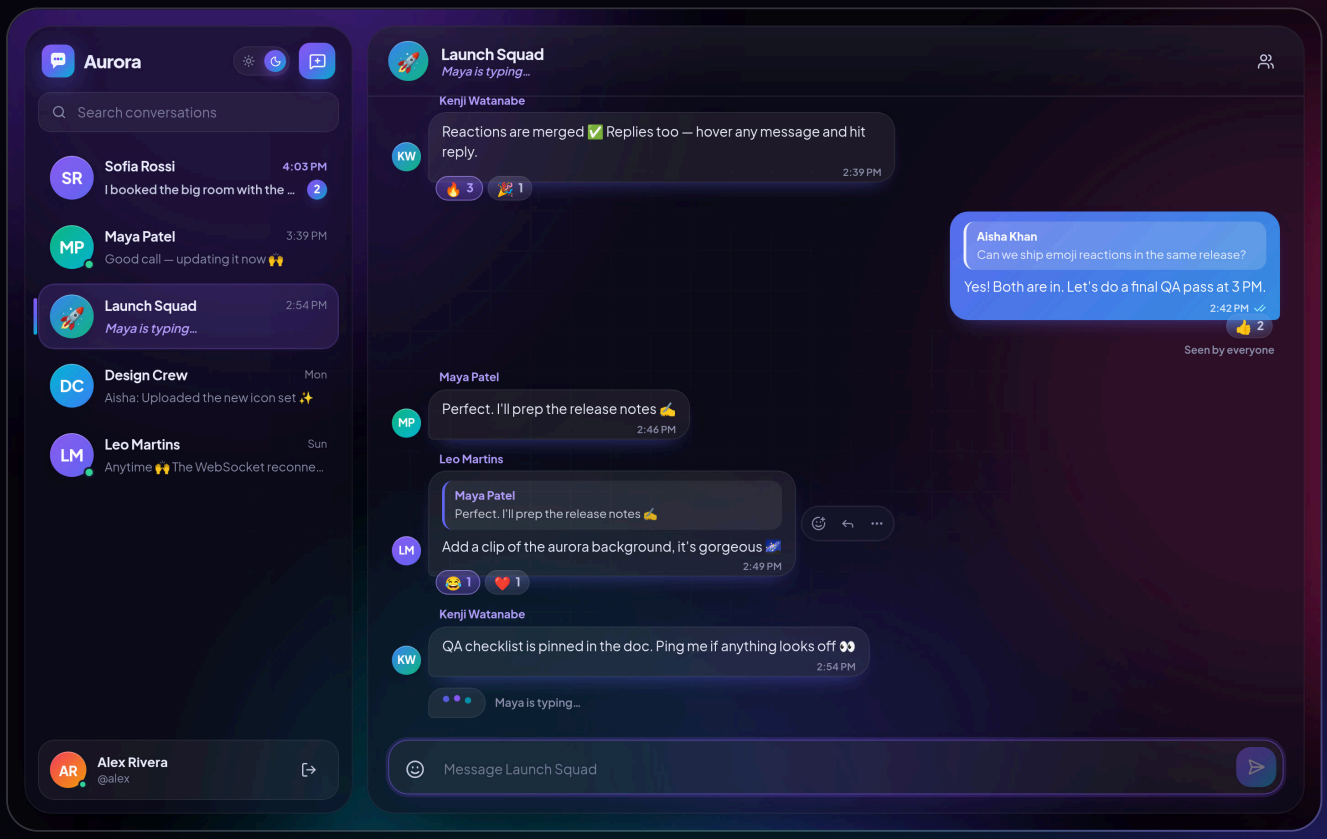
Aurora Chat

Project guide · v1.0

Real-time chat, beautifully in sync.

A full-stack chat app with replies, emoji reactions, presence and a glassy light & dark UI, built with a Java Spring Boot backend and a React frontend, plus a step-by-step guide to hosting it for free.

Java 17+ · Spring Boot 3.5 | React 19 · TypeScript · Vite 8 | STOMP over WebSocket | PostgreSQL / H2 | Vercel + Render + Neon



Contents

01 App tour: screenshots in light & dark mode — features at a glance

02 Architecture — how the pieces talk

03 Project overview (README) — stack, structure, API, configuration

04 Setup & free hosting guide — local run → Neon → Render → Vercel

05 Verification results — automated + end-to-end tests

At a glance

FRONTEND
React 19 + TypeScript
Vite 8, STOMP client, custom glass design system

BACKEND
Spring Boot 3.5 · Java 17+
REST + STOMP WebSocket, Spring Security, JPA

REAL-TIME
~15–60 ms delivery
measured end-to-end between two browsers

DATABASE
H2 locally · PostgreSQL in prod
paste a Neon/Render URL into DATABASE_URL

HOSTING
$0 / month
Vercel (UI) · Render (API) · Neon (DB)

QUALITY
10 tests + 16 E2E checks
all passing; strict TypeScript build

What you can do

⚡ **Chat in real time**
Direct messages and group chats that update instantly, with reconnect & catch-up.

↩️ **Reply to messages**
Quoted previews; click to jump to and highlight the original.

😍 **React with emoji**
Quick reactions, full picker with search, double-click for ❤️, see who reacted.

🟢 **Presence & typing**
Online dots, “last seen”, and live typing indicators.

✅ **Read receipts**
✓ sent, ✓✓ seen, “Seen by everyone”, unread badges.

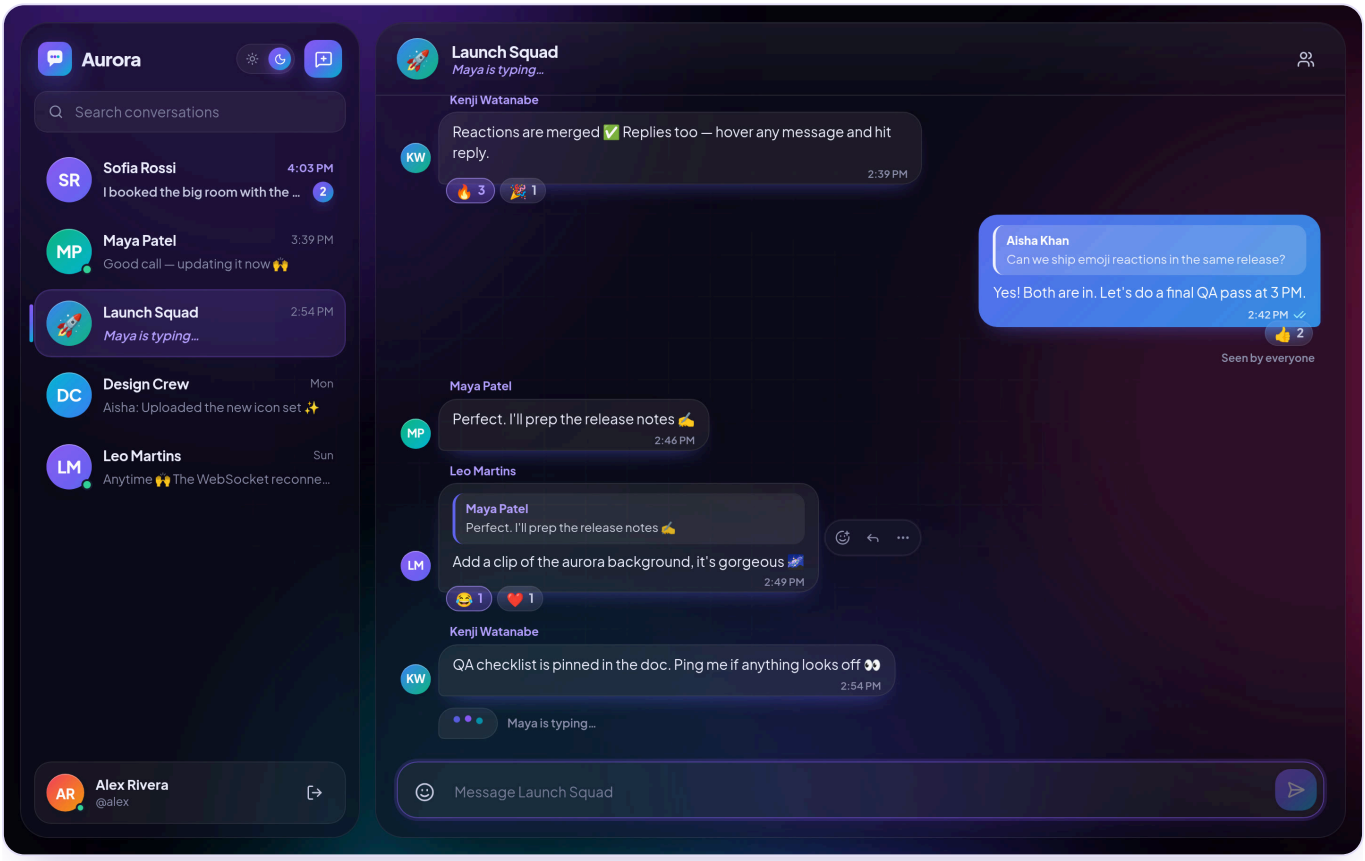
✏️ **Edit & delete**
Fix typos (↑ edits your last message) or remove messages.

🌓 **Light & dark mode**
Follows the OS, remembers your choice, circular reveal animation.

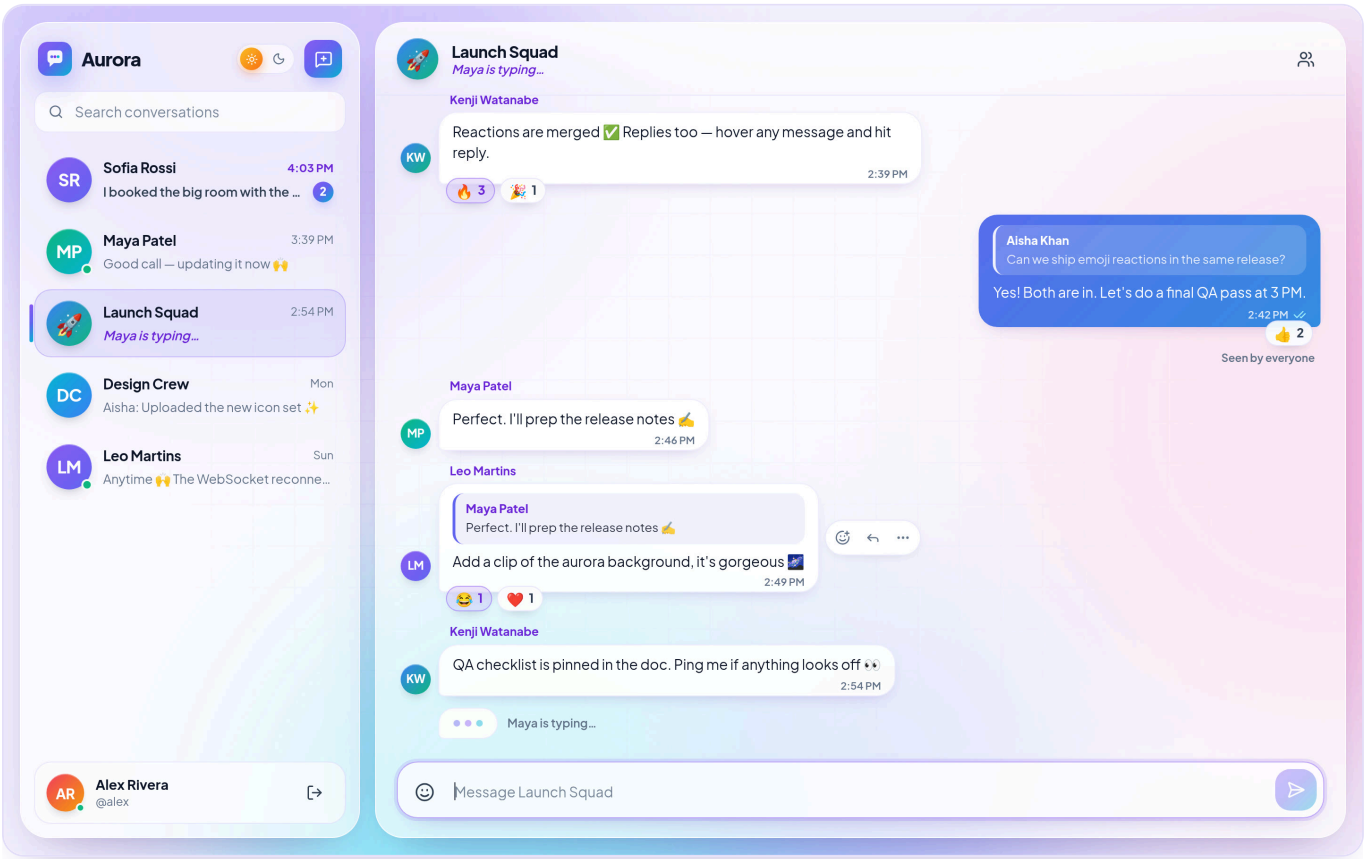
📱 **Responsive**
Desktop two-pane layout, mobile one-pane with back navigation.

Two README files are included in the project: `README.md` (GitHub project page) and `SETUP_AND_DEPLOYMENT.md` (step-by-step setup and free hosting on Vercel). Both are reproduced in this PDF.

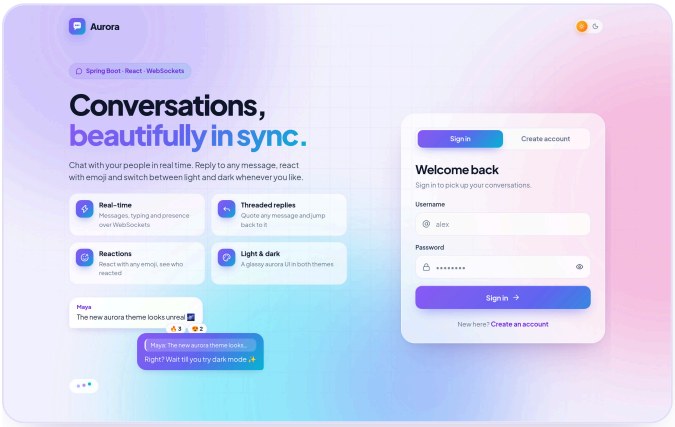
1 · App tour



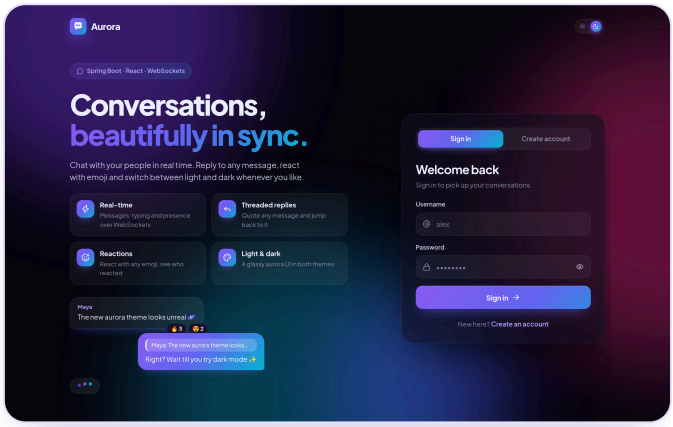
Group chat in dark mode: replies, reactions, read receipts, presence and a live typing indicator



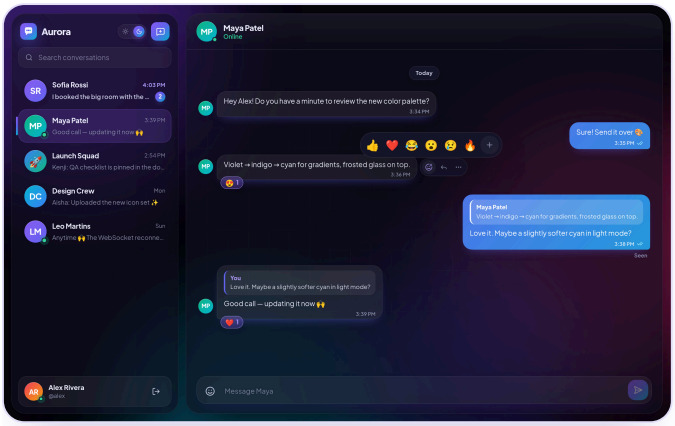
The same conversation in light mode



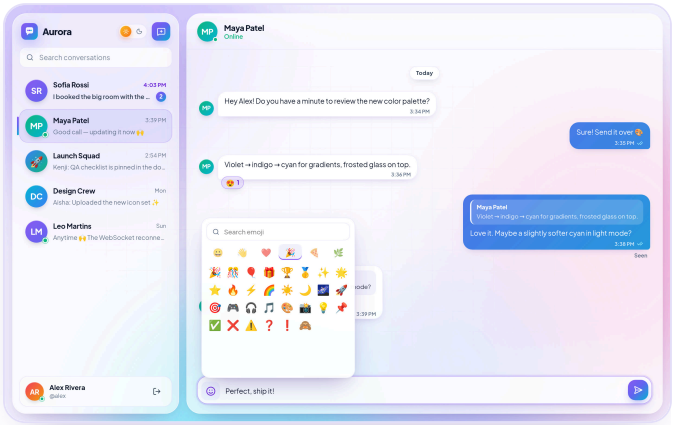
Sign in / create account (light)



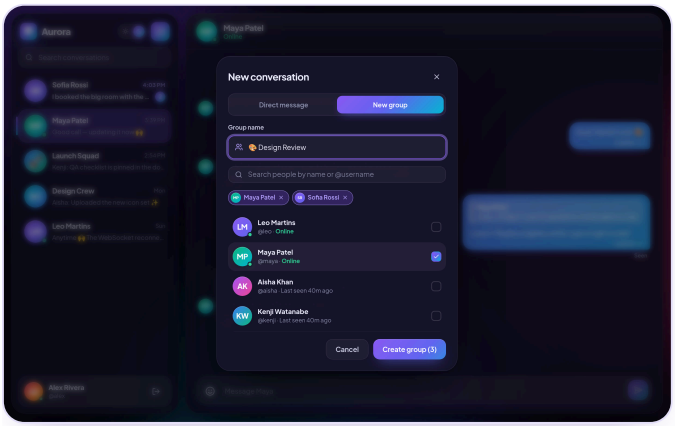
Sign in / create account (dark)



Hover any message: quick reactions, reply and more



Replying with the searchable emoji picker open

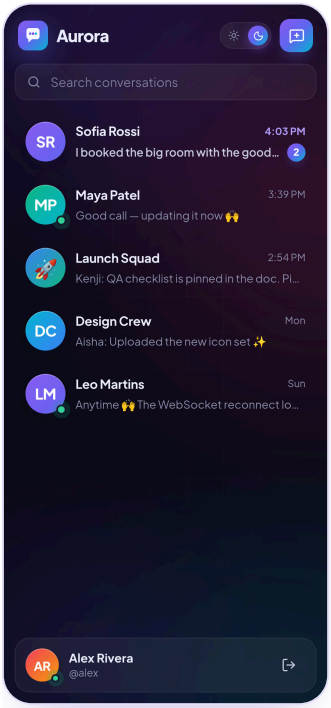


Start a direct message or create a group

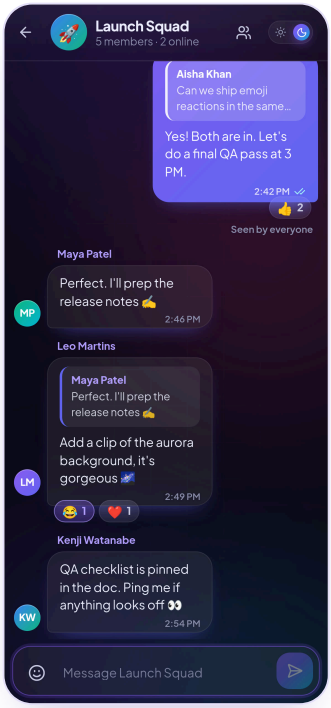
Tips

- Hover a message (tap on phones) for **react · reply · more**.
- **Double-click** a message to ❤️ it.
- Click a quoted reply to **jump** to the original.
- **Enter** sends, **Shift+Enter** adds a line, ↑ edits your last message, **Esc** cancels.

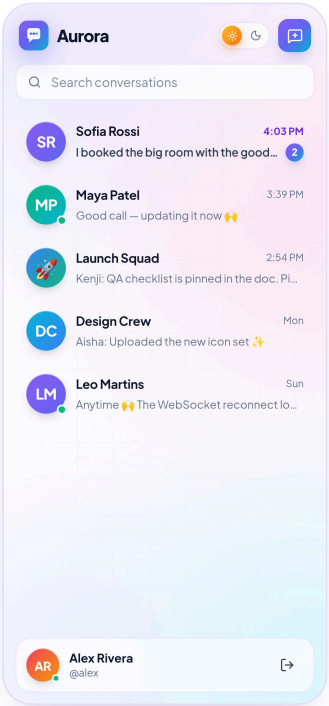
On your phone



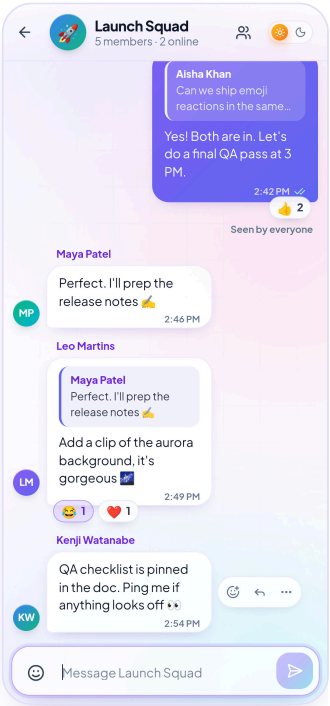
Conversations · dark



Chat · dark

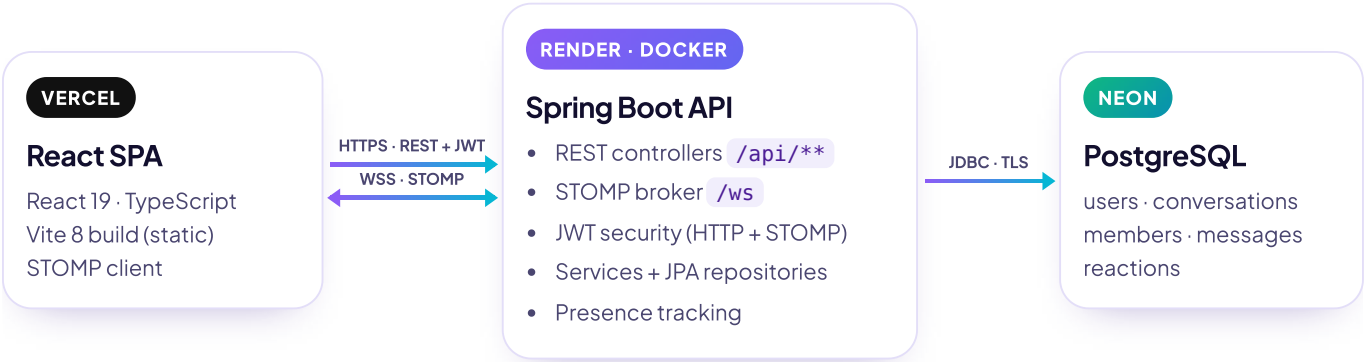


Conversations · light



Chat · light

2 · Architecture



What happens when you send a message

1. **Send** The composer shows the message instantly (optimistic UI) and POSTs it to `/api/conversations/{id}/messages`.
2. **Validate & save** The API checks membership, the reply target and length, then stores it in one transaction.
3. **Push** After commit, a `MESSAGE_CREATED` event goes to every member's private queue `/user/queue/events`.
4. **Receive** Other members' browsers render it immediately; the sender's optimistic copy is reconciled by `clientId`.
5. **Seen** Viewers move their read marker; a `READ` event turns ✓ into ✓✓ for the sender.

Data model

app_users	conversations	conversation_members	messages	message_reactions
id username (unique) display_name password_hash last_seen_at	id type DIRECT|GROUP name direct_key (unique) updated_at	conversation_id user_id joined_at last_read_message_id	id conversation_id sender_id content · reply_to_id edited_at · deleted	message_id user_id emoji (unique together)

Security highlights

- Passwords hashed with **BCrypt**; stateless **JWT** (HS256) for REST and for the STOMP `CONNECT` frame.
- Every conversation, message and reaction request checks **membership**; outsiders get 404.
- Clients may only subscribe to `/user/queue/events` and `/topic/presence`, and only send to `/app/**`, so nobody can inject events into another user's queue.
- **CORS** allow-list (exact origins or patterns) shared by REST and the WebSocket handshake; React escapes all message text.

3 Project overview

From `README.md`, the GitHub project page

✨ Features

Feature	What it does
⚡ **Real-time messaging**	Messages appear instantly for everyone in the conversation (STOMP over WebSocket), with automatic reconnect and catch-up after network drops.
↩ **Replies**	Reply to any message. The reply shows a quoted preview; click it to jump to (and highlight) the original — older history loads automatically.
😍 **Reactions**	One-tap quick reactions, a searchable emoji picker, double-click to ❤️, live counts, and a tooltip showing who reacted.
👥 **Direct & group chats**	Start a 1:1 chat with anyone, or create named groups. New conversations pop up for the other members in real time.
🟢 **Presence & typing**	Online dots, "last seen" times and "Maya is typing…" indicators.
✅ **Read receipts & unread badges**	✓ sent, ✓✓ seen, "Seen by everyone" in groups, unread counters in the sidebar and the tab title.
✏️ **Edit & delete**	Edit or delete your own messages (press ↑ in an empty box to edit your last one).
🌓 **Light & dark mode**	Follows your OS by default, remembers your choice, and switches with a circular reveal animation.
📱 **Responsive**	Feels at home on desktop and phones.
🔐 **Secure by default**	BCrypt passwords, stateless JWT auth for REST *and* WebSocket, per-conversation authorization, locked-down STOMP destinations.

🧱 Tech stack

Layer	Technology
Frontend	React 19, TypeScript, Vite 8, `@stomp/stompjs`, lucide icons, hand-written CSS design system (glassmorphism, CSS variables for theming)
Backend	Java 17+, Spring Boot 3.5 (Web, WebSocket/STOMP, Security, Data JPA, Validation), JJWT
Database	H2 (zero-setup local file DB) · PostgreSQL in production (e.g. free Neon)
Hosting (free)	Frontend on **Vercel** · API on **Render** (Docker) · Postgres on **Neon**

🚀 Quick start (local)

Prerequisites: JDK 17+ and Node.js 20.19+ (or 22+). No database needed.

```
git clone https://github.com/<your-username>/aurora-chat.git
cd aurora-chat

# 1) Backend → http://localhost:8080   (demo users are optional)
cd backend
APP_DEMO_DATA_ENABLED=true ./mvnw spring-boot:run        # Windows CMD: set "APP_DEMO_DATA_ENABLED=true"
&& mvnw.cmd spring-boot:run

# 2) Frontend → http://localhost:5173   (in a second terminal)
cd frontend
cp .env.example .env.local
npm install
npm run dev
```

Open http://localhost:5173, sign in as **alex / password123** (demo data) or create an account. To chat with yourself, open a second **private window** and sign in as **maya / password123**.

> Full step-by-step instructions, including free deployment to Vercel, are in **SETUP_AND_DEPLOYMENT.md**.

📁 Project structure

```
aurora-chat/
├── backend/                          Spring Boot API (Maven)
│   ├── src/main/java/com/aurora/chat/
│   │   ├── auth/                     register / login / me
│   │   ├── chat/                     conversations, messages, reactions, typing, event publisher
│   │   ├── user/                     users, presence tracking
│   │   ├── security/                 JWT service, HTTP filter, STOMP auth interceptor
│   │   ├── config/                   security, WebSocket, DATABASE_URL parsing, demo data
│   │   ├── common/                   DTO mapper, errors, health endpoint
│   │   └── dto/                      request / response records
│   ├── src/test/java/...             REST flow, WebSocket and unit tests
│   └── Dockerfile
├── frontend/                         React + Vite SPA
│   ├── src/components/               UI (sidebar, chat window, bubbles, composer, pickers…)
│   ├── src/context/                  auth, theme, toasts, chat state (reducer + STOMP client)
│   ├── src/lib/                      API client, config, formatting, emoji data
│   └── vercel.json
├── docs/                             PDF guide + screenshots
├── render.yaml                       one-click Render blueprint for the API
├── docker-compose.yml                optional local Postgres + API
├── README.md                         ← you are here
└── SETUP_AND_DEPLOYMENT.md           setup & free hosting guide
```

🔌 API reference

All endpoints except register/login/health require `Authorization: Bearer <token>`. Errors look like `{"status":400,"error":"Bad Request","message":"…"}`.

Method	Endpoint	Description
`POST`	`/api/auth/register`	`{username, displayName, password}` → `{token, user}`
`POST`	`/api/auth/login`	`{username, password}` → `{token, user}`
`GET`	`/api/auth/me`	Current user
`GET`	`/api/users?query=`	Search people (excludes you)
`GET`	`/api/conversations`	Your conversations with last message, unread count, read states
`POST`	`/api/conversations/direct`	`{userId}` → open (or create) a DM
`POST`	`/api/conversations/group`	`{name, memberIds[]}` → create a group
`GET`	`/api/conversations/{id}/messages?before=&limit=`	Page of messages (oldest → newest)
`POST`	`/api/conversations/{id}/messages`	`{content, replyToId?, clientId?}` → send
`POST`	`/api/conversations/{id}/read`	`{messageId}` → move your read marker
`PATCH`	`/api/messages/{id}`	`{content}` → edit your message
`DELETE`	`/api/messages/{id}`	Delete your message
`POST`	`/api/messages/{id}/reactions`	`{emoji}` → toggle your reaction
`GET`	`/api/health`	Liveness (public)

WebSocket (STOMP)

Connect to `ws(s)://<api-host>/ws` with the STOMP header `Authorization: Bearer <token>`.

Direction	Destination	Payload
subscribe	`/user/queue/events`	`{type, conversationId, payload}` where type ∈ `MESSAGE_CREATED`, `MESSAGE_UPDATED`, `REACTIONS_UPDATED`, `CONVERSATION_UPSERTED`, `TYPING`, `READ`
subscribe	`/topic/presence`	`{userId, online, lastSeenAt}`
send	`/app/conversations/{id}/typing`	`{typing: true | false}`

⚙️ Configuration

Backend (environment variables)

Variable	Default	Purpose
`PORT`	`8080`	HTTP port (set automatically by most hosts)

Variable	Default	Purpose
`DATABASE_URL`	*(unset → H2 file DB)*	Postgres URL, e.g. `postgresql://user:pass@host/db?sslmode=require` (Neon/Render format is fine)
`APP_JWT_SECRET`	dev value	**Required in production.** Any long random string
`APP_JWT_EXPIRATION`	`7d`	Token lifetime
`APP_CORS_ALLOWED_ORIGINS`	`http://localhost:5173,…`	Comma-separated frontend origins; patterns like `https://*.vercel.app` allowed
`APP_DEMO_DATA_ENABLED`	`false`	Seed demo users/conversations into an **empty** database
`DB_POOL_SIZE`	`5`	Max DB connections

Frontend (build-time, `frontend/.env.local` locally or Vercel → Environment Variables)

Variable	Example	Purpose
`VITE_API_URL`	`https://aurora-chat-api.onrender.com`	Backend base URL (no trailing slash)
`VITE_WS_URL`	*(optional)*	Override the WebSocket URL (defaults to `VITE_API_URL` + `/ws`)

🧪 Testing

```
cd backend && ./mvnw test          # REST flow, WebSocket delivery/typing/presence/auth, DATABASE_URL
parser
cd frontend && npm run build       # strict TypeScript check + production build
```

The project was also verified end-to-end with two real browsers (sign-in, presence, typing, delivery, replies, jump-to-quote, reactions, edit, delete, read receipts, groups, new DMs, theme toggle, reload and server-restart reconnect).

🛣️ Ideas for next steps

- Image & file attachments (e.g. Cloudinary / S3 free tiers)
- Push notifications (Web Push)
- Message search and pinned messages
- Group management (rename, add/remove members)
- Horizontal scaling with an external STOMP broker (RabbitMQ) and Redis-backed presence

4 Setup & free hosting guide

From `SETUP_AND_DEPLOYMENT.md`: run locally, then deploy to Neon, Render and Vercel

This guide takes you from the downloaded source code to a live app on the internet, **for $0**:

1. How the free hosting fits together
2. Prerequisites
3. Run it locally
4. Push the code to GitHub
5. Create a free PostgreSQL database (Neon)
6. Deploy the Java backend (Render)
7. Deploy the React frontend (Vercel)
8. Connect frontend and backend (CORS)
9. Test your live app
10. Optional extras
11. Troubleshooting

Expect about **30–40 minutes** for a first deployment.

---

1. How the free hosting fits together

```
Browser ──HTTPS──▶  Vercel  (React frontend, static files)
   │
   ├──HTTPS (REST) ──▶  Render  (Spring Boot API, Docker)  ──▶  Neon (PostgreSQL)
   └──WSS (live chat)──▶  Render
```

Piece	Service	Free plan highlights*
React frontend	**Vercel** – Hobby	Global CDN, HTTPS, auto-deploy on every `git push`
Java API + WebSockets	**Render** – Free web service	Docker support, HTTPS, auto-deploy; sleeps after ~15 min without traffic, wakes in ~1 min
Database	**Neon** – Free	Serverless Postgres, data persists, scales to zero when idle

*Free-tier limits change from time to time, so check each provider's pricing page.

> **Why isn't the backend on Vercel too?** Vercel serves static sites and short-lived serverless functions (Node.js, Python, Go, Ruby…). It doesn't run Java, and serverless functions can't keep the WebSocket connections that real-time chat needs. So the React app goes on Vercel, where it's fast and free, and the Spring Boot API goes on Render, which runs Docker containers for free. The frontend just needs to know the API's URL (`VITE_API_URL`).

---

2. Prerequisites

Accounts (sign up with GitHub to save time):

- GitHub
- Vercel
- Render
- Neon

Tools (for running locally):

Tool	Version	Check with
Java JDK	17 or newer (21 recommended), e.g. Temurin	`java -version`
Node.js	20.19+ or 22+ (nodejs.org)	`node -v`
Git	any recent	`git --version`

You don't need to install Maven: the project includes the Maven Wrapper (`./mvnw`). You don't need a database either, because locally the API uses an embedded H2 database.

3. Run it locally

3.1 Get the code

Unzip the project (or clone your repository) and open a terminal in the `aurora-chat` folder.

3.2 Start the backend (port 8080)

```
cd backend
# macOS / Linux — APP_DEMO_DATA_ENABLED seeds demo users the first time
APP_DEMO_DATA_ENABLED=true ./mvnw spring-boot:run
```

Windows PowerShell:

```
cd backend
$env:APP_DEMO_DATA_ENABLED="true"; .\mvnw.cmd spring-boot:run
```

Windows Command Prompt:

```
cd backend
set "APP_DEMO_DATA_ENABLED=true" && mvnw.cmd spring-boot:run
```

The first run downloads dependencies (a few minutes). The backend is ready when you see `Started AuroraChatApplication` . Check http://localhost:8080/api/health; it should show `{"status":"UP",…}` .

> Local data lives in `backend/data/` . Delete that folder to start fresh. Demo data is only added when the database is empty.

3.3 Start the frontend (port 5173)

Open a **second terminal**:

```
cd frontend
cp .env.example .env.local        # Windows: copy .env.example .env.local
npm install
npm run dev
```

Open http://localhost:5173.

3.4 Try it

- Sign in as **alex** / **password123**, or create your own account.
- Open a **private/incognito window**, sign in as **maya** / **password123**, and chat between the two windows. You'll see typing indicators, instant delivery, replies, reactions and read receipts.
- Toggle light/dark mode with the ☀️/🌙 switch.

3.5 Run the tests (optional)

```
cd backend && ./mvnw test        # 10 tests: REST flow, WebSocket events, URL parsing
cd frontend && npm run build     # strict TypeScript check + production build
```

3.6 Production-like run with PostgreSQL (optional, needs Docker)

```
docker compose up --build        # Postgres + API on http://localhost:8080
```

Then run the frontend as in step 3.3.

---

4. Push the code to GitHub

1. On GitHub click **New repository**, name it `aurora-chat`, and leave it **empty** (no README, .gitignore or license, because the project already has them).
2. In the project folder:

```
git init
git add .
git commit -m "Aurora Chat: Spring Boot + React real-time chat"
git branch -M main
git remote add origin https://github.com/<your-username>/aurora-chat.git
git push -u origin main
```

The included `.gitignore` keeps `node_modules`, build output and local databases out of the repo.

---

5. Create a free PostgreSQL database (Neon)

Render's disk is temporary (it's wiped on every deploy/restart), so production data must live in a real database.

1. Go to https://console.neon.tech and click **New Project**.

2. Name it `aurora-chat` and pick a **region close to where you'll run the API**:
   - Render **Singapore** → Neon **AWS Asia Pacific (Singapore)** (good for India/Asia)
   - Render **Oregon** → Neon **AWS US West (Oregon)**
   - Render **Frankfurt** → Neon **AWS Europe (Frankfurt)**
3. Click **Create**. On the project dashboard click **Connect** and copy the **connection string**. It looks like:

```
postgresql://neondb_owner:AbC123xyz@ep-cool-sky-123456.ap-southeast-1.aws.neon.tech/neondb?
sslmode=require&channel_binding=require
```

Keep it handy for the next step. You can paste it as-is: the backend converts this format to JDBC automatically, and tables are created on first start.

> 🔒 Treat the connection string like a password. Never commit it to Git.

---

6. Deploy the Java backend (Render)

You can use the included **Blueprint** (fastest) or create the service manually. Pick one.

Option A — Blueprint (uses `render.yaml`)

1. Render dashboard → **New +** → **Blueprint**.
2. Connect your GitHub account if asked, then select the `aurora-chat` repository.
3. Render reads `render.yaml` and shows the `aurora-chat-api` service. Fill in:
   - `DATABASE_URL` → your Neon connection string
   - `APP_CORS_ALLOWED_ORIGINS` → `https://*.vercel.app` for now (you'll tighten this in step 8)
4. Click **Apply**. `APP_JWT_SECRET` is generated for you.

Option B — Manual web service

1. Render dashboard → **New +** → **Web Service** → select the `aurora-chat` repository.
2. Settings:

Field	Value
Name	`aurora-chat-api` (this becomes part of your URL)
Region	same region as your Neon database
Branch	`main`
Root Directory	`backend`
Language / Runtime	**Docker** (auto-detected from `backend/Dockerfile`)
Instance Type	**Free**

3. **Environment Variables** → add:

Key	Value
`DATABASE_URL`	your Neon connection string
`APP_JWT_SECRET`	click **Generate** (or paste any long random string)
`APP_CORS_ALLOWED_ORIGINS`	`https://*.vercel.app` (tightened in step 8)

4. **Advanced** → **Health Check Path** → `/api/health`.
5. Click **Create Web Service**.

Wait for the first deploy

The first Docker build takes about **5–10 minutes**. Watch the **Logs** tab until you see `Started AuroraChatApplication` and the service shows **Live**.

Copy your URL from the top of the page, e.g. `https://aurora-chat-api.onrender.com`, and open `https://aurora-chat-api.onrender.com/api/health` in your browser. You should see `{"status":"UP",…}`. 🎉

> 💤 On the free plan the service sleeps after ~15 minutes without traffic. The next request wakes it in about a minute, and the frontend shows a friendly "Waking up the server…" banner while that happens.

---

7. Deploy the React frontend (Vercel)

1. Go to https://vercel.com/new and **import** your `aurora-chat` GitHub repository.
2. Configure the project:

Field	Value
Project Name	`aurora-chat` (your site becomes `aurora-chat.vercel.app` if available)
Framework Preset	**Vite** (auto-detected)
Root Directory	click **Edit** → choose **`frontend`** ⚠️ important
Build Command	`npm run build` (default, also set in `vercel.json`)
Output Directory	`dist` (default)

3. Open **Environment Variables** and add:

Key	Value
`VITE_API_URL`	your Render URL, e.g. `https://aurora-chat-api.onrender.com` (**no trailing slash**)

4. Click **Deploy**. After about a minute you get a URL like `https://aurora-chat.vercel.app`.

> `VITE_*` variables are baked in at **build time**. If you change `VITE_API_URL` later, redeploy: **Deployments** → ⋯ → **Redeploy**.

---

8. Connect frontend and backend (CORS)

Browsers only let your site call the API if the API allows your site's **origin**.

1. Copy your Vercel production URL, e.g. `https://aurora-chat.vercel.app`.
2. Render → `aurora-chat-api` → **Environment** → edit `APP_CORS_ALLOWED_ORIGINS`:

   ```
   https://aurora-chat.vercel.app
   ```

   To also allow Vercel preview deployments and a custom domain, use a comma-separated list:

   ```
   https://aurora-chat.vercel.app,https://aurora-chat-*.vercel.app,https://chat.yourdomain.com
   ```

3. **Save Changes**. Render redeploys automatically in a minute or two.

Rules: include `https://`, no trailing slash, no paths. The same list protects both the REST API and the WebSocket handshake.

---

9. Test your live app

1. Open your Vercel URL. If the API was asleep you'll see "Waking up the server…" for up to a minute.
2. **Create an account** (username + display name + password).
3. Open a private window (or use your phone), create a second account, and start a chat with **New conversation** → pick the other user.
4. Try everything: typing indicator, replies (hover a message → ↩), reactions (hover → 😊+, or double-click a message for ❤️), edit/delete, a group chat and dark mode.

Share the URL with friends and they can sign up and chat with you right away.

---

10. Optional extras

Demo content in production

Set `APP_DEMO_DATA_ENABLED=true` on Render **before the first start against an empty database** to get the demo users. ⚠️ Everyone can sign in to those accounts with `password123`, so turn it back off and don't use it for a real community.

Keep the API awake

Free services sleep when idle. If you want instant responses, ping `https://<your-api>.onrender.com/api/health` every 10–14 minutes with a free monitor such as UptimeRobot or cron-job.org. Keep in mind this uses your free instance hours (one always-on free service fits within Render's monthly free hours at the time of writing).

Custom domain

- Frontend: Vercel → Project → **Settings → Domains** → add `chat.yourdomain.com` and follow the DNS instructions.
- Add the new origin to `APP_CORS_ALLOWED_ORIGINS` on Render.

- Optional: a custom domain for the API (Render → **Settings → Custom Domains**), then update `VITE_API_URL` on Vercel and redeploy.

Automatic deploys

Both Vercel and Render redeploy on every push to `main`. GitHub Actions (`.github/workflows/ci.yml`) builds and tests both apps on every push and pull request.

Other free backend hosts

The backend is a standard Docker image (`backend/Dockerfile`), so it also runs on Koyeb, Google Cloud Run (free tier, needs billing enabled) or an Oracle Cloud Always Free VM. Set the same environment variables (`DATABASE_URL`, `APP_JWT_SECRET`, `APP_CORS_ALLOWED_ORIGINS`) and make sure the host supports WebSockets.

---

11. Troubleshooting

Symptom	Cause & fix
Red banner **"Backend URL not configured"**	`VITE_API_URL` wasn't set when Vercel built the site. Add it (Project → Settings → Environment Variables) and **redeploy**.
"Waking up the server…" never goes away	Open the API's `/api/health` directly. If it doesn't load, check the Render **Logs**: usually a typo in `DATABASE_URL` or the build failed.
Browser console: **CORS error** / `No 'Access-Control-Allow-Origin'`	`APP_CORS_ALLOWED_ORIGINS` doesn't match your site exactly. Use `https://your-app.vercel.app` (no trailing slash), save, and wait for the redeploy.
Sidebar stuck on **"Connecting…"**	The WebSocket can't connect. It uses the same CORS list, and the API URL must be `https://` (it becomes `wss://`). Also check that the API is awake.
Signed out after a redeploy	`APP_JWT_SECRET` changed or the database was reset. Sign in again. Keep the secret stable.
All data disappears after each deploy	`DATABASE_URL` isn't set, so the API fell back to the temporary H2 file. Set the Neon URL (step 6).
Render logs: `password authentication failed` / SSL errors	Re-copy the Neon connection string. Special characters in a password must be URL-encoded (e.g. `@` → `%40`). Keep `sslmode=require`.
Render build fails	Check that Root Directory is `backend` (manual setup) or that `render.yaml` is at the repo root (Blueprint). Look for the first `ERROR` line in the build logs.
Vercel build fails with `tsc` errors	Make sure **Root Directory** is `frontend`, then run `npm run build` locally to see the same error.
Login says **"Invalid username or password"**	Usernames are case-insensitive; passwords are not. Demo accounts only exist if demo data was enabled on an empty database.
Port 8080 or 5173 already in use (local)	Stop the other process, or run the backend with `PORT=8081` and set `VITE_API_URL=http://localhost:8081` in `frontend/.env.local`.

5 · Verification results

Everything below ran against the real Spring Boot server and the production frontend build before delivery.

Automated backend tests: `./mvnw test`

Suite	What it covers	Result
ChatFlowIntegrationTest (3)	Register/login validation, auth required, people search, DM create-once, send with clientId echo, unread counts, replies, reaction toggling, read markers, edit/delete permissions, outsider 404s, groups, pagination	**PASS**
WebSocketIntegrationTest (3)	Real STOMP clients: conversation, typing, message and reaction push; presence online/offline; rejects bad tokens; blocks forbidden subscriptions	**PASS**
DatabaseUrlParserTest (4)	Neon/Render URLs → JDBC, SSL defaults, URL-encoded passwords, JDBC passthrough	**PASS**

End-to-end with two real browsers (Chromium)

Check	Result
Sign in two users, unread badge visible	**PASS**
Presence: “Online” shown for the other user	**PASS**
Typing indicator reaches the other browser	**PASS**
Real-time delivery (measured 14–59 ms)	**PASS**
Reply with quoted preview · jump-to-quote highlight	**PASS**
Reaction from hover bar · double-click ❤️ and toggle off	**PASS**
Edit own message (“edited” label) · delete own message	**PASS**
Read receipt “Seen”	**PASS**
Group delivery + unread badge · new DM appears for the recipient	**PASS**
Theme toggle · no console errors	**PASS**
Session restored after reload · auto-reconnect and delivery after a server restart	**PASS**

Production build: strict TypeScript (`tsc -b`) + Vite: about 97 KB of gzipped JavaScript. The backend Docker image is tuned for 512 MB free instances.